Object Detection

Finding what is in an image and where, from bounding boxes and IoU to YOLO, Faster R-CNN and DETR, custom training, evaluation, tracking and deployment.

From the basics to advanced techniques, with explanations and working code.

1. **Foundations**: detection versus classification, bounding box formats, drawing boxes, IoU
2. **How detectors work**: anchors, two-stage and one-stage detectors, non-maximum suppression, loss functions
3. **Evaluation**: matching predictions, precision and recall, AP and mAP
4. **Using and training detectors**: YOLO, Faster R-CNN, DETR, custom datasets and training
5. **Advanced**: deployment, tracking, distance measurement, open-vocabulary detection and segmentation

Contents

Part 1: Foundations

1. What is object detection?
2. Bounding boxes and their formats
3. Drawing boxes on an image
4. Intersection over Union (IoU)

Part 2: How detectors work

5. From sliding windows to anchors
6. Families of detectors
7. Non-maximum suppression (NMS)
8. What detectors learn: the loss functions

Part 3: Evaluation

9. Matching predictions to the ground truth
10. Average precision (AP) and mAP

Part 4: Using and training detectors

11. Choosing a model
12. YOLO in three lines (Ultralytics)
13. Faster R-CNN with torchvision
14. DETR: detection with a Transformer
15. Building a custom dataset
16. Training a custom YOLO model

Part 5: Advanced topics

17. Deployment: export and speed
18. Tracking objects across video frames
19. Measuring distance between detected people
20. Beyond fixed classes: open-vocabulary detection and segmentation

Summary and interview questions

Cheat sheet

Interview questions to practise

Part 1: Foundations

1. What is object detection?

Computer vision tasks answer increasingly detailed questions about an image:

Task	Question	Output
Classification	What is in this image?	One label: "dog"
Localisation	What is it, and where?	One label and one box
Object detection	What objects are there, and where is each one?	Any number of (label, box, confidence) triples
Semantic segmentation	Which class does each pixel belong to?	A class for every pixel
Instance segmentation	Which pixels belong to each individual object?	A mask per object

Detection is everywhere: self-driving cars (pedestrians, signs, vehicles), retail (shelf monitoring, checkout-free stores), manufacturing (defect detection), medicine (tumours in scans), security, agriculture (fruit counting), sports analytics and accessibility apps.

2. Bounding boxes and their formats

A bounding box is the smallest axis-aligned rectangle around an object. It takes four numbers, but different tools write those numbers differently. **Mixing formats up is the most common bug in detection code.**

Format	Numbers	Used by
xyxy	x_min, y_min, x_max, y_max (pixels)	Pascal VOC, torchvision, most maths
xywh	x_min, y_min, width, height (pixels)	COCO annotations
cxcywh, normalised	centre x, centre y, width, height, each divided by the image size (0–1)	YOLO label files

Image coordinates start at the **top-left** corner: x grows to the right and y grows **downwards**.

Python

Tested: output below is real

```python
import numpy as np

def xyxy_to_xywh(b):
    x1, y1, x2, y2 = b
    return [x1, y1, x2 - x1, y2 - y1]

def xywh_to_xyxy(b):
    x, y, w, h = b
    return [x, y, x + w, y + h]

def xyxy_to_yolo(b, img_w, img_h):
    x1, y1, x2, y2 = b
    return [round((x1 + x2) / 2 / img_w, 4), round((y1 + y2) / 2 / img_h, 4),
            round((x2 - x1) / img_w, 4), round((y2 - y1) / img_h, 4)]

def yolo_to_xyxy(b, img_w, img_h):
    cx, cy, w, h = b
    return [round((cx - w / 2) * img_w), round((cy - h / 2) * img_h),
            round((cx + w / 2) * img_w), round((cy + h / 2) * img_h)]

box = [100, 50, 300, 250]                # xyxy in a 640 x 480 image
print("xyxy:", box)
print("xywh (COCO):", xyxy_to_xywh(box))
print("YOLO:", xyxy_to_yolo(box, 640, 480))
print("Round trip back to xyxy:", yolo_to_xyxy(xyxy_to_yolo(box, 640, 480), 640, 480))
```

Output

```
xyxy: [100, 50, 300, 250]
xywh (COCO): [100, 50, 200, 200]
YOLO: [0.3125, 0.3125, 0.3125, 0.4167]
Round trip back to xyxy: [100, 50, 300, 250]
```

3. Drawing boxes on an image

You will draw boxes constantly: to check annotations, debug predictions and present results. OpenCV makes it easy. (OpenCV stores colours as **BGR**, not RGB.)

Python

Tested: output below is real

```python
import cv2

# A synthetic scene: a "car", a "person" and a "ball" on a grey background
scene = np.full((300, 480, 3), 225, dtype=np.uint8)
cv2.rectangle(scene, (40, 150), (230, 250), (90, 90, 200), -1)       # car body
cv2.circle(scene, (85, 255), 22, (40, 40, 40), -1); cv2.circle(scene, (190, 255), 22, (40, 40, 40),
-1)
cv2.rectangle(scene, (300, 90), (345, 260), (160, 110, 60), -1)      # person body
cv2.circle(scene, (322, 70), 22, (150, 180, 220), -1)                # person head
cv2.circle(scene, (420, 245), 25, (50, 170, 240), -1)                # ball

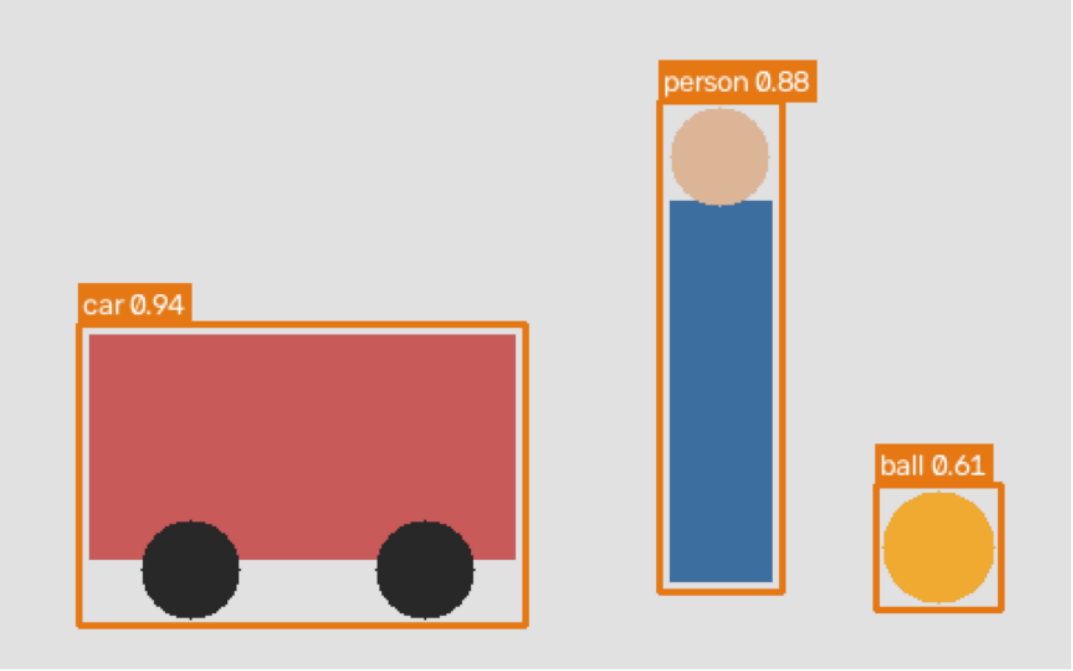
detections = [("car", 0.94, [35, 145, 235, 280]), ("person", 0.88, [295, 45, 350, 265]),
              ("ball", 0.61, [392, 217, 448, 273])]

def draw_detections(image, detections):
    out = image.copy()
    for label, score, (x1, y1, x2, y2) in detections:
        cv2.rectangle(out, (x1, y1), (x2, y2), (20, 120, 230), 2)
        text = f"{label} {score:.2f}"
        (tw, th), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, 0.45, 1)
        cv2.rectangle(out, (x1, y1 - th - 6), (x1 + tw + 4, y1), (20, 120, 230), -1)
        cv2.putText(out, text, (x1 + 2, y1 - 4), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1)
    return out

annotated = draw_detections(scene, detections)
cv2.imwrite("detections.jpg", annotated)
print("Saved detections.jpg with", len(detections), "boxes")
```

Output

```
Saved detections.jpg with 3 boxes
```



The output of draw_detections: each object gets a box, a class label and a confidence score.

4. Intersection over Union (IoU)

How do we measure whether a predicted box matches the true box? **IoU** divides the area where the boxes overlap by the area they cover together. It ranges from 0 (no overlap) to 1 (identical boxes). IoU is used everywhere in detection: in evaluation, in non-maximum suppression, in training targets and in tracking.

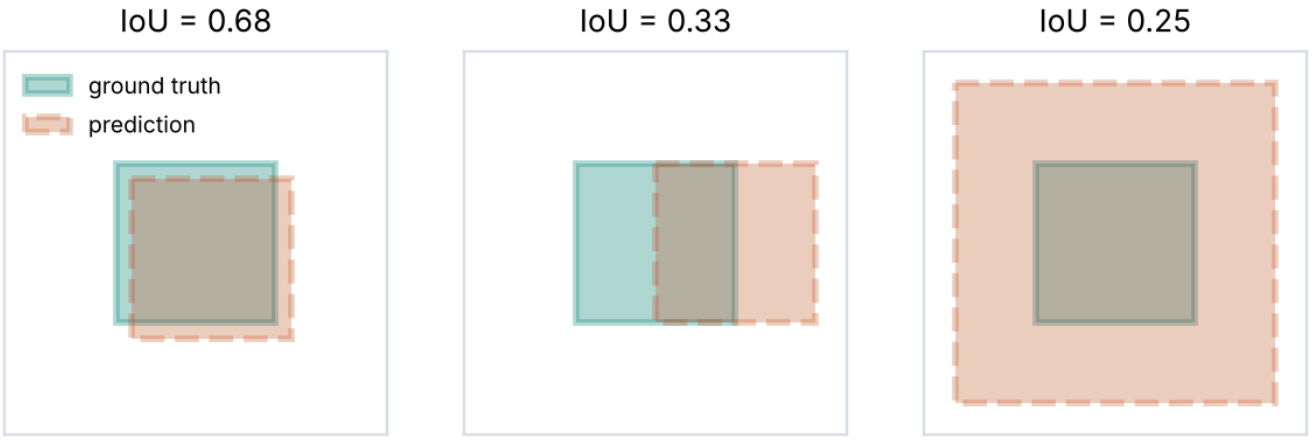
Python — Tested: output below is real

```python
def iou(a, b):
    """IoU of two boxes in xyxy format."""
    inter_w = max(0, min(a[2], b[2]) - max(a[0], b[0]))
    inter_h = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    intersection = inter_w * inter_h
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return intersection / (area_a + area_b - intersection)

truth = [100, 100, 200, 200]
for name, pred in [("identical", [100, 100, 200, 200]), ("shifted a little", [110, 110, 210, 210]),
                   ("shifted half", [150, 100, 250, 200]), ("too big", [50, 50, 250, 250]),
                   ("no overlap", [300, 300, 400, 400])]:
    print(f"{name:17s} IoU = {iou(truth, pred):.3f}")
```

Output

```
identical         IoU = 1.000
shifted a little  IoU = 0.681
shifted half      IoU = 0.333
too big           IoU = 0.250
no overlap        IoU = 0.000
```



IoU drops quickly as boxes drift apart. A prediction shifted by only 10% of the box size already scores just 0.68, and a box twice the true size scores 0.25.

> **Key idea:** an IoU of **0.5** is the traditional threshold for "this prediction found the object" (Pascal VOC). COCO evaluates at ten thresholds from 0.5 to 0.95, rewarding precise boxes.

Part 2: How detectors work

5. From sliding windows to anchors

The naive approach is to slide a window of every size across the image and run a classifier at each position. That means millions of classifier runs per image, which is far too slow.

Modern detectors make every prediction in **one pass** of a convolutional network (or Transformer). The network produces a grid of feature vectors, and at each grid cell it predicts boxes relative to reference shapes called **anchors**. An anchor is a template box of a given size and aspect ratio (tall for people, wide for cars). The network only has to predict small *adjustments* to each anchor, plus a class score.

Python — Tested: output below is real

```python
def make_anchors(feature_size, stride, sizes=(32, 64, 128), ratios=(0.5, 1.0, 2.0)):
    anchors = []
    for row in range(feature_size):
        for col in range(feature_size):
            cx, cy = (col + 0.5) * stride, (row + 0.5) * stride      # centre of this grid cell
            for size in sizes:
                for ratio in ratios:                                 # ratio = height / width
                    w, h = size / np.sqrt(ratio), size * np.sqrt(ratio)
                    anchors.append([cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2])
    return np.array(anchors)

anchors = make_anchors(feature_size=20, stride=32)      # a 640 x 640 image, 20 x 20 grid
print("Anchors:", anchors.shape, "(grid cells x sizes x aspect ratios)")
print("Three anchors at the first cell (xyxy):")
print(anchors[[0, 1, 2]].round(1))
```

Output

```
Anchors: (3600, 4) (grid cells x sizes x aspect ratios)
Three anchors at the first cell (xyxy):
[[-6.6  4.7 38.6 27.3]
 [ 0.   0.  32.  32. ]
 [ 4.7 -6.6 27.3 38.6]]
```

Thousands of anchors cover every position, size and shape. Since YOLOv8, many detectors are **anchor-free**: they predict the distances from each grid point to the four box edges directly, which removes the need to hand-tune anchor shapes.

6. Families of detectors

Family	How it works	Examples	Trade-off
Two-stage	Stage 1 proposes regions likely to contain objects; stage 2 classifies and refines each one	R-CNN (2014), Fast R-CNN, **Faster R-CNN** (2015), Mask R-CNN	Very accurate, especially for small objects; slower
One-stage	Predicts classes and boxes for every grid cell in a single pass	**YOLO** (2016 onwards), SSD, RetinaNet	Real-time speed; now matches two-stage accuracy
Transformer-based	A Transformer attends over image features and outputs a fixed set of object predictions directly	**DETR** (2020), Deformable DETR, RT-DETR	No anchors or NMS; elegant; RT-DETR is real-time

YOLO ("You Only Look Once") is the most widely used family in industry, because it's fast, accurate and easy to train. The Ultralytics library provides recent versions behind a three-line API (Section 12).

7. Non-maximum suppression (NMS)

A detector typically outputs *many* overlapping boxes for the same object, because neighbouring grid cells and anchors all fire. **NMS** cleans this up:

1. Sort all boxes by confidence.
2. Keep the highest-scoring box.
3. Remove every remaining box that overlaps it with IoU above a threshold (often 0.5–0.7).
4. Repeat with the next highest remaining box.

NMS is run separately for each class, so a person and a bicycle that overlap aren't suppressed.

Python

Tested: output below is real

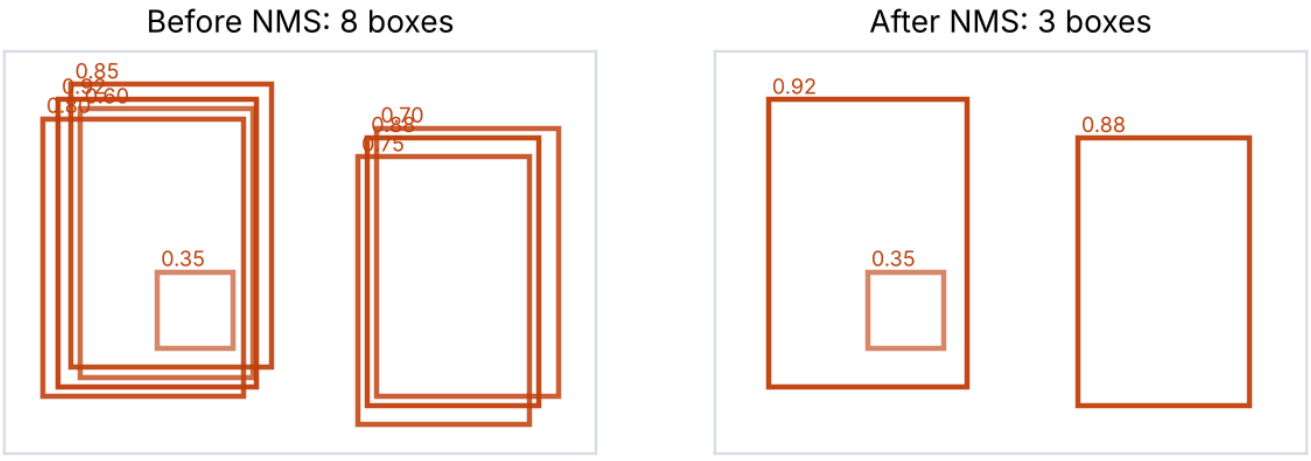
```python
def nms(boxes, scores, iou_threshold=0.5):
    order = list(np.argsort(-np.asarray(scores)))
    keep = []
    while order:
        best = order.pop(0)
        keep.append(int(best))
        order = [i for i in order if iou(boxes[best], boxes[i]) < iou_threshold]
    return keep

raw_boxes = [[48, 60, 152, 210], [55, 52, 160, 200], [40, 70, 145, 215], [60, 65, 150, 205],   # person A
             [210, 80, 300, 220], [205, 90, 295, 230], [215, 75, 310, 215],                     # person B
             [100, 150, 140, 190]]                                                              # a stray box
raw_scores = [0.92, 0.85, 0.80, 0.60, 0.88, 0.75, 0.70, 0.35]

kept = nms(raw_boxes, raw_scores, iou_threshold=0.5)
print(f"{len(raw_boxes)} raw boxes -> {len(kept)} after NMS: indices {kept}")
for i in kept:
    print(f"  box {raw_boxes[i]}  score {raw_scores[i]}")
```

Output

```
8 raw boxes -> 3 after NMS: indices [0, 4, 7]
  box [48, 60, 152, 210]  score 0.92
  box [210, 80, 300, 220]  score 0.88
  box [100, 150, 140, 190]  score 0.35
```



NMS keeps the most confident box for each object and removes its overlapping duplicates. The low-confidence stray box survives NMS because it doesn't overlap the others much; a confidence threshold removes boxes like that.

> **Watch out:** NMS can wrongly remove a *real* object that heavily overlaps another, for example people in a crowd. Soft-NMS lowers overlapping scores instead of deleting them, and DETR-style models avoid NMS entirely.

8. What detectors learn: the loss functions

A detector is trained with several losses added together:

- **Classification loss**: is there an object here, and which class? This is usually cross-entropy, or **focal loss**, which down-weights the huge number of easy background examples so training focuses on the hard ones.
- **Box regression loss**: how far is the predicted box from the true box? Older detectors used L1 distances on the coordinates. Modern ones optimise IoU directly, using **GIoU**, **DIoU** or **CIoU**, which still give a useful signal when the boxes don't overlap at all. Plain IoU is 0 for every non-overlapping pair, so it can't tell the model which way to move.

Python — Tested: output below is real

```python
def giou(a, b):
    """Generalised IoU: IoU minus the fraction of the enclosing box not covered by either box."""
    inter = max(0, min(a[2], b[2]) - max(a[0], b[0])) * max(0, min(a[3], b[3]) - max(a[1], b[1]))
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    enclose = (max(a[2], b[2]) - min(a[0], b[0])) * (max(a[3], b[3]) - min(a[1], b[1]))
    return inter / union - (enclose - union) / enclose

truth = [100, 100, 200, 200]
for name, pred in [("overlapping", [150, 100, 250, 200]), ("near, no overlap", [210, 100, 310, 200]),
                   ("far, no overlap", [400, 100, 500, 200])]:
    print(f"{name:17s} IoU = {iou(truth, pred):.3f}   GIoU = {giou(truth, pred):+.3f}")
```

Output

```
overlapping       IoU = 0.333   GIoU = +0.333
near, no overlap  IoU = 0.000   GIoU = -0.048
far, no overlap   IoU = 0.000   GIoU = -0.500
```

Both non-overlapping predictions have IoU 0, but GIoU correctly scores the nearer one as better, which gives the model a direction to improve in.

Part 3: Evaluation

9. Matching predictions to the ground truth

To evaluate a detector, each prediction is labelled a **true positive** (TP) or a **false positive** (FP):

1. Sort predictions by confidence, highest first.
2. For each prediction, find the unmatched ground-truth box of the same class with the highest IoU.
3. If that IoU is at least the threshold (for example 0.5), it's a **TP**, and the ground-truth box is now used up.
4. Otherwise it's an **FP**: a duplicate, a badly placed box, or a hallucinated object.
5. Ground-truth boxes that are never matched are **false negatives** (missed objects).

Then **precision** = TP / (TP + FP) (how many detections were right) and **recall** = TP / (all ground-truth objects) (how many objects were found).

Python — Tested: output below is real

```python
# Ground truth for one class ("person") across three images
ground_truth = {
    "img1": [[50, 50, 150, 200], [200, 60, 280, 210]],
    "img2": [[30, 40, 120, 190]],
    "img3": [[100, 80, 180, 230], [250, 70, 330, 220], [400, 90, 470, 240]],
}
# Predictions: (image, confidence, box)
predictions = [
    ("img1", 0.95, [52, 55, 148, 198]),   ("img3", 0.91, [102, 85, 178, 225]),
    ("img2", 0.88, [35, 45, 125, 185]),   ("img1", 0.80, [205, 65, 285, 205]),
    ("img3", 0.72, [255, 60, 335, 215]),  ("img1", 0.65, [55, 52, 152, 202]),    # duplicate of img1's
first person
    ("img3", 0.55, [500, 100, 560, 240]), ("img2", 0.40, [200, 100, 260, 200]),  # nothing there
    ("img3", 0.35, [380, 60, 450, 200]),                                         # poorly placed box
]

def match(predictions, ground_truth, iou_threshold=0.5):
    used = {img: [False] * len(boxes) for img, boxes in ground_truth.items()}
    results = []
    for img, conf, box in sorted(predictions, key=lambda p: -p[1]):
        ious = [iou(box, gt) for gt in ground_truth[img]]
        best = int(np.argmax(ious)) if ious else -1
        if best >= 0 and ious[best] >= iou_threshold and not used[img][best]:
            used[img][best] = True
            results.append((conf, True))
        else:
            results.append((conf, False))
    return results

results = match(predictions, ground_truth)
n_truth = sum(len(b) for b in ground_truth.values())
tp = np.cumsum([r[1] for r in results])
fp = np.cumsum([not r[1] for r in results])
precision, recall = tp / (tp + fp), tp / n_truth

print(" conf  result  precision  recall")
for (conf, ok), p, r in zip(results, precision, recall):
    print(f" {conf:.2f}  {'TP' if ok else 'FP':6s}  {p:9.2f}  {r:6.2f}")
```

Output

```
 conf  result  precision  recall
 0.95  TP           1.00    0.17
 0.91  TP           1.00    0.33
 0.88  TP           1.00    0.50
 0.80  TP           1.00    0.67
 0.72  TP           1.00    0.83
 0.65  FP           0.83    0.83
 0.55  FP           0.71    0.83
 0.40  FP           0.62    0.83
 0.35  FP           0.56    0.83
```

Reading down the table is like *lowering* the confidence threshold: each extra prediction can raise recall (finding more objects) but risks lowering precision (more mistakes).

10. Average precision (AP) and mAP

The precision–recall pairs above trace a **precision–recall curve**. **Average precision (AP)** summarises that curve as one number: the area under it, after making precision monotonically decreasing (at each recall level, use the best precision achievable at that recall or higher). AP is computed per class; **mAP** is the

mean of AP across classes.

Python — Tested: output below is real

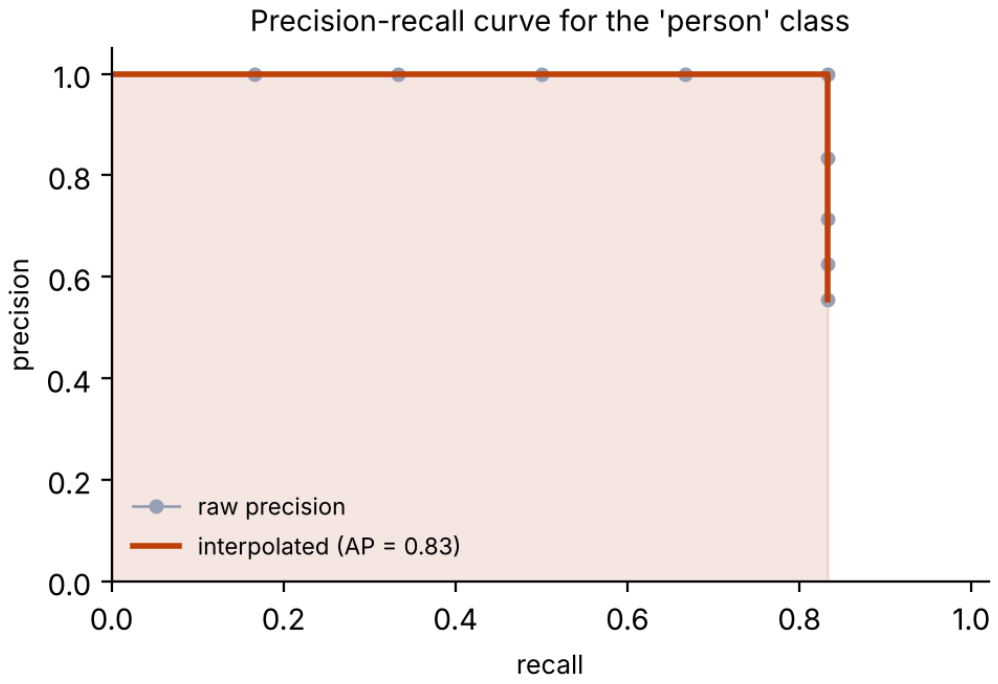
```python
def average_precision(precision, recall):
    """All-point interpolated AP (Pascal VOC 2010+ and similar to COCO)."""
    r = np.concatenate([[0], recall, [1]])
    p = np.concatenate([[1], precision, [0]])
    for i in range(len(p) - 2, -1, -1):          # make precision monotonically decreasing
        p[i] = max(p[i], p[i + 1])
    changes = np.where(r[1:] != r[:-1])[0]       # sum rectangle areas where recall changes
    return float(np.sum((r[changes + 1] - r[changes]) * p[changes + 1]))

ap50 = average_precision(precision, recall)
print(f"AP@0.5 for 'person': {ap50:.3f}")

# COCO-style: average AP over IoU thresholds 0.50, 0.55, ..., 0.95
aps = []
for t in np.arange(0.5, 0.96, 0.05):
    res = match(predictions, ground_truth, iou_threshold=t)
    tp_t = np.cumsum([x[1] for x in res]); fp_t = np.cumsum([not x[1] for x in res])
    aps.append(average_precision(tp_t / (tp_t + fp_t), tp_t / n_truth))
print("AP at each IoU threshold:", np.round(aps, 2))
print(f"AP@[0.5:0.95] (the COCO 'mAP'): {np.mean(aps):.3f}")
```

Output

```
AP@0.5 for 'person': 0.833
AP at each IoU threshold: [0.83 0.83 0.83 0.83 0.83 0.83 0.83 0.33 0.17 0.  ]
AP@[0.5:0.95] (the COCO 'mAP'): 0.633
```



The shaded area under the interpolated curve is the AP. The curve stops short of recall 1.0 because one person was never detected.

AP drops sharply at strict IoU thresholds: the boxes found the people, but not all of them precisely. That's exactly what COCO's AP@[0.5:0.95] is designed to reward.

Interview tip: when someone quotes "mAP", ask *which* one. **mAP@0.5** (also written mAP50) is lenient about box placement; **mAP@0.5:0.95** (COCO mAP) is strict and is the standard headline number in papers and in YOLO's training output.

Part 4: Using and training detectors

11. Choosing a model

Need	Good starting point
Real-time detection, easy training, edge devices	**YOLO** (Ultralytics), nano or small size
Highest accuracy, offline processing	Larger YOLO models, RT-DETR, or a two-stage model such as Faster R-CNN
Research or full PyTorch control	**torchvision** detection models
No fixed class list: "find anything I name"	Open-vocabulary detectors: OWLv2, Grounding DINO, YOLO-World (Section 19)

12. YOLO in three lines (Ultralytics)

Python — Needs: pip install ultralytics

```python
from ultralytics import YOLO

model = YOLO("yolo11n.pt")          # nano model pre-trained on COCO's 80 classes; downloads once
results = model("https://ultralytics.com/images/bus.jpg", conf=0.25)

r = results[0]
for box, conf, cls in zip(r.boxes.xyxy.tolist(), r.boxes.conf.tolist(), r.boxes.cls.tolist()):
    print(f"{model.names[int(cls)]:10s} {conf:.2f}  {[round(v) for v in box]}")

r.save(filename="bus_detected.jpg")   # image with boxes drawn
```

Useful prediction options: `conf` (minimum confidence), `iou` (the NMS threshold), `classes=[0]` (only people), `imgsz` (input size). Video files and webcams work the same way:

Python — Needs: pip install ultralytics

```python
for r in model.predict(source="traffic.mp4", stream=True, classes=[2, 5, 7]):   # car, bus, truck
    print(len(r.boxes), "vehicles in this frame")
# source=0 uses the default webcam
```

13. Faster R-CNN with torchvision

Python — Needs: pip install torch torchvision

```python
from torchvision.io import read_image
from torchvision.models.detection import (FasterRCNN_ResNet50_FPN_V2_Weights,
                                          fasterrcnn_resnet50_fpn_v2)
from torchvision.utils import draw_bounding_boxes
from torchvision.transforms.functional import to_pil_image

weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn_v2(weights=weights, box_score_thresh=0.7).eval()

image = read_image("street.jpg")                      # uint8 tensor (3, H, W)
batch = [weights.transforms()(image)]                 # the model's own preprocessing
prediction = model(batch)[0]                          # dict: boxes (xyxy), labels, scores

labels = [weights.meta["categories"][i] for i in prediction["labels"]]
drawn = draw_bounding_boxes(image, boxes=prediction["boxes"], labels=labels, colors="red", width=3)
to_pil_image(drawn).save("street_detected.jpg")
```

14. DETR: detection with a Transformer

DETR (DEtection TRansformer) treats detection as *set prediction*. A CNN extracts image features; a Transformer encoder-decoder with a fixed number of learned **object queries** (100 by default) outputs one prediction per query, either an object or "no object". During training, each query is matched one-to-one to a ground-truth object (Hungarian matching), so duplicate predictions are penalised and **no NMS is needed**.

Python — Needs: pip install transformers torch timm pillow

```python
from transformers import pipeline

detector = pipeline("object-detection", model="facebook/detr-resnet-50")
for obj in detector("street.jpg", threshold=0.9):
    print(f"{obj['label']:12s} {obj['score']:.2f}  {obj['box']}")
# box is {'xmin': ..., 'ymin': ..., 'xmax': ..., 'ymax': ...}
```

15. Building a custom dataset

To detect your own objects (hard hats, defects, products, plant diseases), you need labelled images.

1. **Collect images** that look like real deployment conditions: lighting, angles, distances, backgrounds. Diversity matters more than volume.
2. **Label** them with a tool such as **CVAT**, **Label Studio** or **Roboflow**. Draw tight boxes, and label *every* instance; an unlabelled object teaches the model that it's background.
3. **Split** into train, validation and test sets (for example 70/20/10). Keep near-duplicate images (consecutive video frames) in the same split, or your validation scores will be inflated.

YOLO expects one `.txt` label file per image, with one line per object: `class cx cy w h`, normalised to 0–1.

```
datasets/hardhats/
├── data.yaml
├── images/
│   ├── train/   site_001.jpg  site_002.jpg ...
│   └── val/     site_101.jpg ...
└── labels/
    ├── train/   site_001.txt  site_002.txt ...
    └── val/     site_101.txt ...
```

YAML

```yaml
# data.yaml
path: datasets/hardhats
train: images/train
val: images/val
names:
  0: helmet
  1: head        # a person without a helmet
```

Many datasets come in **COCO JSON** format instead. Converting them uses the box-format functions from Section 2:

Python — Tested: output below is real

```python
coco = {   # a minimal COCO-format annotation file
    "images": [{"id": 1, "file_name": "site_001.jpg", "width": 640, "height": 480}],
    "categories": [{"id": 1, "name": "helmet"}, {"id": 2, "name": "head"}],
    "annotations": [
        {"image_id": 1, "category_id": 1, "bbox": [120, 40, 60, 50]},    # COCO bbox = xywh in pixels
        {"image_id": 1, "category_id": 2, "bbox": [400, 60, 55, 70]},
    ],
}

class_index = {c["id"]: i for i, c in enumerate(coco["categories"])}   # COCO ids -> 0, 1, ...
for img in coco["images"]:
    lines = []
    for ann in coco["annotations"]:
        if ann["image_id"] == img["id"]:
            cx, cy, w, h = xyxy_to_yolo(xywh_to_xyxy(ann["bbox"]), img["width"], img["height"])
            lines.append(f"{class_index[ann['category_id']]} {cx:.4f} {cy:.4f} {w:.4f} {h:.4f}")
    print(img["file_name"].replace(".jpg", ".txt"), "->")
    print("\n".join(lines))
```

Output

```
site_001.txt ->
0 0.2344 0.1354 0.0938 0.1042
1 0.6680 0.1979 0.0859 0.1458
```

16. Training a custom YOLO model

Always start from **pre-trained weights** (transfer learning). The model already knows edges, textures and shapes from COCO, so a few hundred labelled images per class can give good results.

Python — Needs: pip install ultralytics

```python
from ultralytics import YOLO

model = YOLO("yolo11n.pt")                      # start from COCO-pretrained weights
model.train(data="datasets/hardhats/data.yaml", epochs=50, imgsz=640, batch=16, patience=15)

metrics = model.val()                            # evaluate on the validation set
print("mAP50:   ", metrics.box.map50)
print("mAP50-95:", metrics.box.map)

best = YOLO("runs/detect/train/weights/best.pt")
best("new_site_photo.jpg")[0].save(filename="check.jpg")
```

The training run saves loss curves, precision–recall curves, a confusion matrix and sample predictions in `runs/detect/train/`. Check them, not just the final number.

Training tips

Problem	What to try
Validation mAP much lower than training mAP	More varied data, stronger augmentation, a smaller model, early stopping
Small objects missed	Larger input size (`imgsz=1024`), tiling large images, more examples of small objects
One class much worse than others	More examples of it; check its labels for mistakes and consistency
Many false positives on background	Add *background images* with no objects (empty label files)
Works in testing, fails in deployment	The training images don't match deployment conditions; collect data from the real setting

YOLO's training pipeline applies augmentation automatically: **mosaic** (stitching four images into one), random scaling, flips and colour jitter. This is a big part of why it trains well on small datasets.

Part 5: Advanced topics

17. Deployment: export and speed

Training happens in PyTorch, but deployment often needs a different runtime:

Python — Needs: pip install ultralytics

```python
model = YOLO("runs/detect/train/weights/best.pt")
model.export(format="onnx")        # ONNX Runtime: CPUs, servers, cross-platform
model.export(format="engine")      # TensorRT: fastest on NVIDIA GPUs (needs TensorRT installed)
model.export(format="coreml")      # Apple devices
model.export(format="tflite")      # Android and edge devices
```

Lever	Effect
Model size (nano → extra-large)	Bigger models are more accurate and slower
Input resolution	Higher resolution finds small objects but costs roughly quadratically more compute
Precision (FP32 → FP16 → INT8)	Half or 8-bit precision speeds up inference considerably, with a small accuracy loss
Batching	Processing several frames together raises throughput on GPUs

18. Tracking objects across video frames

Detection works frame by frame and doesn't know that the person in frame 10 is the same person in frame 11. **Tracking** assigns a persistent ID to each object. The simplest tracker matches each new detection to the existing track it overlaps most:

Python — Tested: output below is real

```python
def iou_tracker(frames, iou_threshold=0.3):
    tracks, next_id, history = {}, 1, []
    for detections in frames:
        assigned = {}
        unmatched = list(range(len(detections)))
        # Greedily match existing tracks to the detections they overlap most
        for track_id, last_box in tracks.items():
            if not unmatched:
                break
            scores = [iou(last_box, detections[i]) for i in unmatched]
            best = int(np.argmax(scores))
            if scores[best] >= iou_threshold:
                assigned[unmatched.pop(best)] = track_id
        for i in unmatched:                              # unmatched detections start new tracks
            assigned[i] = next_id
            next_id += 1
        tracks = {assigned[i]: detections[i] for i in range(len(detections))}
        history.append({tid: box for tid, box in tracks.items()})
    return history

frames = [   # two people walking right, a third appears in frame 3
    [[10, 50, 60, 150], [200, 60, 250, 160]],
    [[25, 52, 75, 152], [212, 58, 262, 158]],
    [[40, 50, 90, 150], [225, 61, 275, 161], [400, 40, 450, 140]],
    [[55, 51, 105, 151], [238, 60, 288, 160], [390, 42, 440, 142]],
]
for t, frame_tracks in enumerate(iou_tracker(frames), start=1):
    print(f"frame {t}: " + "  ".join(f"ID{tid}@x={box[0]}" for tid, box in
sorted(frame_tracks.items())))
```

Output

```
frame 1: ID1@x=10  ID2@x=200
frame 2: ID1@x=25  ID2@x=212
frame 3: ID1@x=40  ID2@x=225  ID3@x=400
frame 4: ID1@x=55  ID2@x=238  ID3@x=390
```

Each person keeps their ID as they move, and the newcomer gets a new one. Production trackers add a **Kalman filter** to predict where each object will be next (SORT), appearance features to re-identify objects after they're hidden (DeepSORT), and smarter matching of low-confidence detections (**ByteTrack**). Ultralytics includes trackers:

Python — Needs: pip install ultralytics

```python
for r in model.track(source="street.mp4", persist=True, tracker="bytetrack.yaml", stream=True):
    if r.boxes.id is not None:
        print(r.boxes.id.int().tolist())   # persistent track IDs in this frame
```

Tracking enables **counting** (people entering a shop, cars crossing a line), measuring dwell time, and analysing trajectories.

19. Measuring distance between detected people

Detections become useful when you compute something from them. A social-distancing monitor, for example, detects people, takes a reference point for each one (the bottom-centre of the box, where their feet touch the ground) and flags pairs that are too close:

Python — Tested: output below is real

```python
from itertools import combinations

people = {1: [40, 80, 90, 260], 2: [110, 90, 160, 265], 3: [300, 70, 350, 250], 4: [520, 85, 570,
262]}
PIXELS_PER_METRE = 60            # from calibration: a known distance measured in the image
MIN_DISTANCE_M = 1.5

feet = {pid: np.array([(b[0] + b[2]) / 2, b[3]]) for pid, b in people.items()}
for a, b in combinations(feet, 2):
    metres = np.linalg.norm(feet[a] - feet[b]) / PIXELS_PER_METRE
    flag = "TOO CLOSE" if metres < MIN_DISTANCE_M else "ok"
    print(f"person {a} - person {b}: {metres:4.1f} m  {flag}")
```

Output

```
person 1 - person 2:  1.2 m  TOO CLOSE
person 1 - person 3:  4.3 m  ok
person 1 - person 4:  8.0 m  ok
person 2 - person 3:  3.2 m  ok
person 2 - person 4:  6.8 m  ok
person 3 - person 4:  3.7 m  ok
```

> **Watch out:** a fixed pixels-per-metre ratio is only valid if the camera looks straight down. With an angled camera, people further away look smaller and closer together. Real systems first apply a **perspective (bird's-eye) transform** with `cv2.getPerspectiveTransform`, using four ground points with known real-world positions, and then measure distances in that top-down view.

20. Beyond fixed classes: open-vocabulary detection and segmentation

Classic detectors only know the classes they were trained on. **Open-vocabulary detectors** combine vision with language (as CLIP does in the Embeddings guide), so you can describe what to find in plain text, with no training:

Python — Needs: pip install transformers torch pillow

```python
from transformers import pipeline

detector = pipeline("zero-shot-object-detection", model="google/owlv2-base-patch16-ensemble")
results = detector("warehouse.jpg", candidate_labels=["forklift", "safety vest", "cardboard box"])
for r in results:
    print(f"{r['label']:14s} {r['score']:.2f}  {r['box']}")
```

Related models worth knowing:

- **Grounding DINO** and **YOLO-World**: stronger open-vocabulary detectors; YOLO-World runs in real time.
- **Mask R-CNN** and **YOLO-seg** models: instance segmentation, giving a pixel mask for each object (in Ultralytics, load a `-seg` model such as `yolo11n-seg.pt`).
- **SAM** (Segment Anything): segments any object from a point or box prompt. A common pipeline is to detect with a box model, then refine each box into a precise mask with SAM.

Summary and interview questions

Cheat sheet

Concept	One-line summary
Object detection	Find every object: class, box and confidence
Box formats	xyxy (VOC, torchvision), xywh (COCO), normalised cxcywh (YOLO)
IoU	Overlap area divided by combined area; 0.5 is the classic match threshold
Anchors	Template boxes the network adjusts; many modern detectors are anchor-free
Two-stage vs one-stage	Propose-then-classify (Faster R-CNN) versus a single dense pass (YOLO)
DETR	A Transformer predicts a set of objects directly; no anchors or NMS
NMS	Keep the best box, remove overlapping duplicates, repeat
Focal loss, GIoU	Focus on hard examples; give a useful box gradient even without overlap
Precision / recall	Share of detections that are right / share of objects found
AP, mAP	Area under the precision–recall curve; averaged over classes (and IoU thresholds for COCO)
Transfer learning	Start from pre-trained weights; fine-tune on your labelled images
Tracking	Keep a consistent ID per object across frames (SORT, DeepSORT, ByteTrack)
Open-vocabulary	Detect objects named in free text, without retraining

Interview questions to practise

1. **What's the difference between detection and segmentation?** Detection gives a box per object; segmentation gives a pixel-level mask, either per class (semantic) or per object (instance).
2. **Explain IoU and where it's used.** Overlap over union of two boxes. It's used to match predictions to ground truth in evaluation, to remove duplicates in NMS, as a training loss (GIoU, CIoU) and to associate detections across frames in tracking.
3. **How does NMS work, and when does it fail?** Keep the highest-scoring box, suppress boxes overlapping it above a threshold, repeat. It fails in crowded scenes where real objects overlap heavily; Soft-NMS or DETR-style models help.
4. **What is mAP@0.5:0.95?** The mean, over classes, of average precision averaged across IoU thresholds from 0.5 to 0.95 in steps of 0.05. It rewards both finding objects and placing boxes precisely.
5. **Compare YOLO and Faster R-CNN.** YOLO is one-stage, predicting boxes densely in one pass: fast and real-time. Faster R-CNN proposes regions and then classifies each: traditionally more accurate on small objects, but slower. Modern YOLO versions have closed most of the accuracy gap.
6. **Your model performs well in validation but badly in production. Why?** Usually a data mismatch: different cameras, lighting, angles or object sizes. Also check for leakage between training and

validation (near-duplicate frames), and label quality. Fix it by collecting and labelling data from the real deployment setting.

7. **How would you build a system that counts people entering a shop?** Detect people with a fast model (YOLO), track them with ByteTrack to get stable IDs, define a virtual line at the entrance, and count each track ID once when its box centre crosses the line in the inward direction.